Episode Alerts

Mobile Application Project Report

Project Title:	Episode Alerts Mobile App
Technology Stack:	React Native, Expo, TypeScript
Prepared By:	Nandu Krishna M 25CSE2015
Prepared For:	Object Oriented Software Engineering Lab
Date:	April 16, 2026

Contents

Chapter 1

SRS

1.1 Introduction

1.1.1 Purpose

This software requirements specification describes the functional and non-functional requirements for Episode Alerts, a cross-platform mobile app that helps users discover television shows, manage a personal watchlist, and stay updated on upcoming episodes. It is intended for developers, testers, reviewers, and future maintainers who need a clear understanding of what the system should do and the limits it must work within.

1.1.2 Scope

Episode Alerts is built with React Native, Expo, and TypeScript, and it uses the TMDB API as its primary content source. The app supports show discovery, detailed show and season views, watchlist management, episode notifications, user preferences, offline-friendly caching, and optional cloud synchronization. It is designed as a companion app rather than a streaming platform.

1.1.3 Definitions, Acronyms, and Abbreviations

API Application Programming Interface.

AsyncStorage
Local key-value storage used to persist app data on the device.

DFD Data Flow Diagram.

Expo The framework and toolchain used to build and run the app.

Firebase
Cloud platform used for optional authentication and data synchronization.

FR Functional Requirement.

SRS Software Requirements Specification.

TMDB The Movie Database, the external service used for show data.

UI User Interface.

1.1.4 References

- The Movie Database API documentation.
- Expo documentation for routing, notifications, and runtime services.
- React Native documentation.
- Firebase documentation for authentication and Firestore.
- Project repository source code and configuration files.

1.1.5 Overview

This SRS is organized into three main parts. The introduction explains the purpose and scope of the product, the overall description summarizes the product context and main characteristics, and the specific requirements section covers the external interfaces, functional behavior, performance expectations, constraints, and quality attributes.

1.2 Overall Description

1.2.1 Product Perspective

Episode Alerts is a single-user mobile app with a modular, service-oriented architecture. Navigation is handled through Expo Router, with tab-based screens for Home, Search, Watchlist, and Settings, while modal and detail screens support deeper navigation flows. The app depends on TMDB for content data, AsyncStorage for local persistence, and optional Firebase services for cloud synchronization.

1.2.2 Product Function

At a high level, the product provides the following functions:

- Discover popular, top-rated, and currently airing television shows.
- Search for shows by title and view detailed information.
- Display season and episode data, including next and last episode details.
- Let users add, remove, and manage shows in a watchlist.
- Notify users about upcoming episodes and release events.

- Store user preferences, analytics settings, and cached content.
- Synchronize selected user data to the cloud when enabled.

1.2.3 User Characteristics

The primary users are everyday TV viewers who want a simple way to follow multiple shows without manually tracking air dates. Most users are expected to have little or no technical background, so the app needs to stay easy to navigate, visually clear, and responsive. More advanced users may rely more heavily on search, filters, sorting, and settings.

1.2.4 General Constraints

The system is constrained by the Expo and React Native runtime, the availability of the TMDB API, and platform permission requirements for notifications and, where enabled, cloud features. The app must run on Android, iOS, and web with shared code, and it must keep sensitive values such as API keys outside the source tree. Offline behavior is limited to locally cached data and stored user content.

1.2.5 Assumptions and Dependencies

The system assumes that TMDB remains available and that the user has network access whenever fresh show data is needed. It also assumes that device permissions can be granted for notifications and that local storage is available for caching and preferences. Optional Firebase features depend on valid configuration and a signed-in user account.

1.3 Specific Requirements

1.3.1 External Interface Requirements

1.3.1.1 User Interfaces

The user interface shall provide a clean tab-based experience with Home, Search, Watchlist, and Settings screens. Detail pages shall present show metadata, season information, episode lists, and countdown information in a readable layout. The interface shall support light, dark, and system themes, show loading and error states, and remain usable across different screen sizes.

1.3.1.2 Hardware Interfaces

The application shall support touch input on mobile devices and pointer or keyboard input on web. It shall use device storage for local persistence and, when notifications are enabled, the operating system notification subsystem. If calendar or device-specific sharing features are added in future updates, the app shall request the relevant platform permissions before access.

1.3.1.3 Software Interfaces

The application shall integrate with the TMDB API for all show, season, and episode data. It shall use AsyncStorage for local caching and user preference storage, Expo Notifications for reminders, Expo Network for connectivity awareness, and optional Firebase authentication and Firestore for cloud synchronization. The app shall also work with React Navigation and Expo Router for screen transitions.

1.3.1.4 Communication Interfaces

All external data exchange shall use secure HTTPS requests. The app shall communicate with TMDB through REST endpoints and shall use Firebase network services only when cloud sync is configured and the user is signed in. Notification interactions shall be handled through the platform notification response channel so tapping a reminder can open the related show.

1.3.2 Functional Requirements

1.3.2.1 Model 1: Discovery and Search

FR-1.1 The system shall load and display popular, top-rated, and currently airing TV shows from TMDB.

FR-1.2 The system shall allow the user to search for TV shows by title or keyword.

FR-1.3 The system shall present a featured show on the home screen and allow the user to refresh the content.

FR-1.4 The system shall use cached data when live data is unavailable so the user can still view previously loaded content.

1.3.2.2 Model 2: Show Details and Seasons

FR-2.1 The system shall display show metadata such as overview, rating, status, genres, and network information.

FR-2.2 The system shall display season information and the episodes available for each season.

FR-2.3 The system shall show next episode and last episode details when the information is available from TMDB.

FR-2.4 The system shall present episode countdown and episode details in a dedicated view.

1.3.2.3 Model 3: Watchlist Management

FR-3.1 The system shall allow the user to add a show to the watchlist from discovery or detail screens.

FR-3.2 The system shall allow the user to remove a show from the watchlist.

FR-3.3 The system shall persist the watchlist locally so it is available after the app restarts.

FR-3.4 The system shall preserve watch progress and watch history for shows in the watchlist.

FR-3.5 The system shall support sorting and filtering of watchlist content where applicable in the user interface.

1.3.2.4 Model 4: Notifications and Preferences

FR-4.1 The system shall let the user enable or disable episode notifications.

FR-4.2 The system shall request notification permissions before scheduling reminders.

FR-4.3 The system shall schedule notifications for upcoming episodes when notifications are enabled.

FR-4.4 The system shall allow the user to change theme, analytics, and notification preferences.

FR-4.5 The system shall open the relevant show when the user taps a notification.

1.3.2.5 Model 5: Synchronization and Analytics

FR-5.1 The system shall support optional cloud synchronization of watchlist and preference data.

FR-5.2 The system shall synchronize local changes to the cloud when the user is signed in and cloud sync is available.

FR-5.3 The system shall record screen views and selected user actions when analytics are enabled.

FR-5.4 The system shall respect the user's analytics preference and stop collecting events when analytics are disabled.

1.3.3 Performance Requirements

The application should remain responsive while data is loaded or synchronized in the background. Cached content should appear quickly when available, and common actions such as adding or removing a show from the watchlist should finish without noticeable delay. Notification synchronization and cloud synchronization should not block the main user interface.

1.3.4 Design Constraints

The implementation shall remain within the React Native, Expo, and TypeScript stack already used by the project. The system shall rely on TMDB as the primary content source and shall not require direct changes to third-party services. Local persistence shall continue to use AsyncStorage, and cloud synchronization shall remain optional so the app can still work without Firebase configuration.

1.3.5 Attributes

The system shall prioritize usability, reliability, portability, maintainability, and security. The interface should be easy to understand for non-technical users, data should persist consistently across sessions, and the codebase should stay modular so new features can be added without major restructuring.

1.3.6 Other Attributes

The application should support offline use where cached data is available, preserve user privacy by keeping analytics opt-in, and remain practical for everyday mobile use. Future work may extend support for more detailed personalization, additional calendar integration, or broader cloud features without changing the core product goals.

Chapter 2

Design

2.1 DFD

Episode Alerts is built around a few core data paths: show discovery and search through TMDB, watchlist and preference storage on the device, episode reminders through the notification layer, and optional cloud sync through Firebase.

2.1.1 Level 0 Context DFD

This level shows Episode Alerts as a single system interacting with the user and its external services.

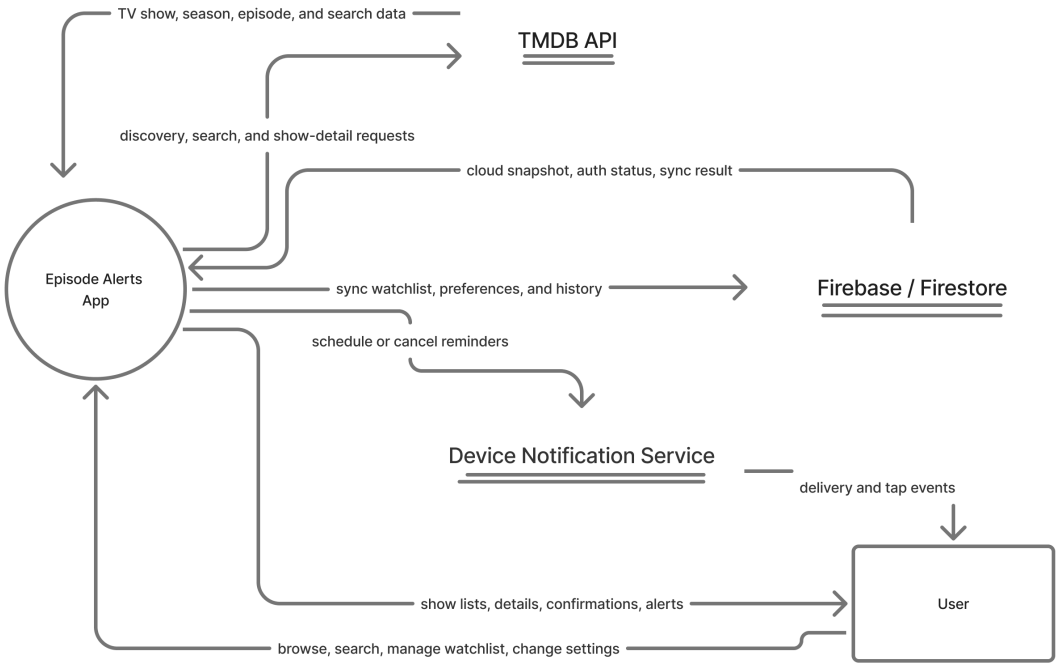


Figure 2.1: Level 0 context DFD for Episode Alerts.

2.1.2 Level 1 Logical DFD

This level breaks the app into its main logical processes and the data stores they use.

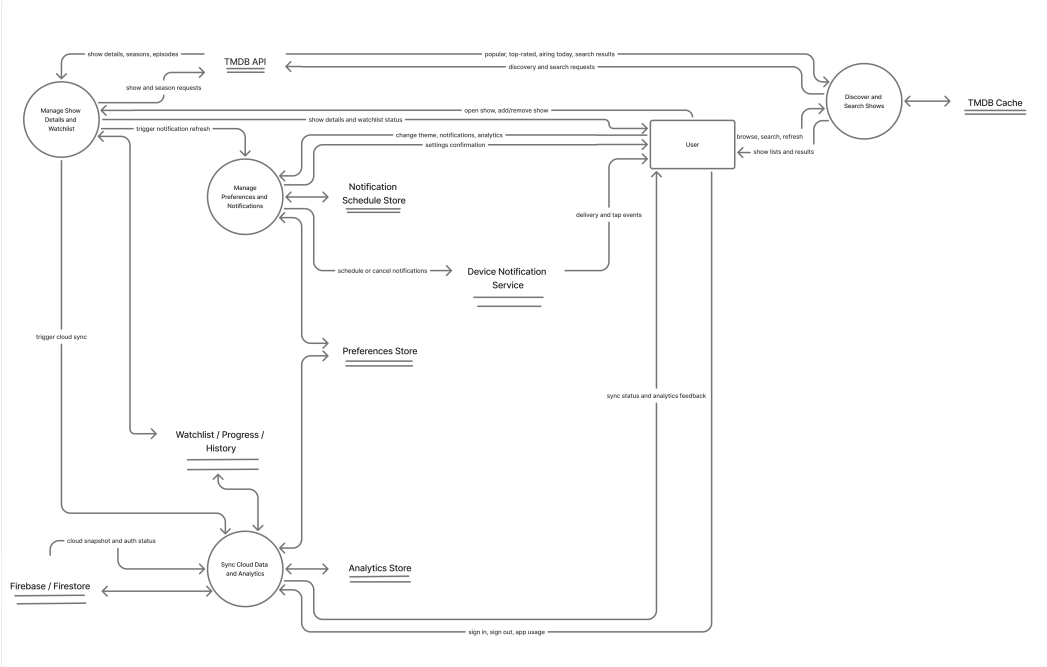


Figure 2.2: Level 1 logical DFD showing the primary processes and data stores.

2.1.3 Level 2 DFD: Content Discovery and Search

This level decomposes the content discovery path because it is the main entry point into the app and the clearest place to show TMDBService, cache hydration, request normalization, and stale-data fallback. It is intentionally more detailed than level 1.

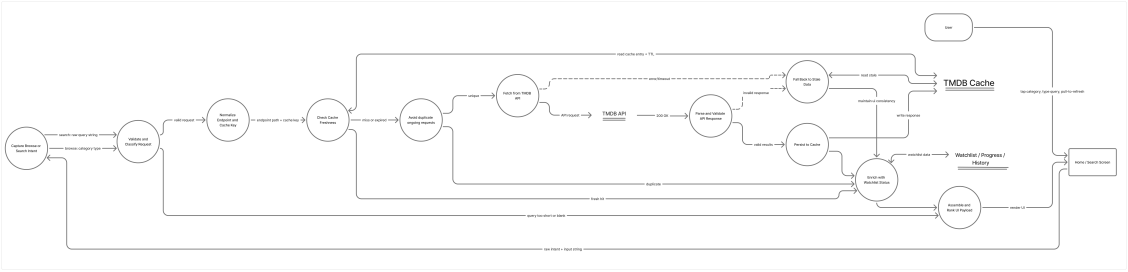


Figure 2.3: Level 2 DFD for content discovery and search, including cache and stale-data fallback paths.

2.2 Use Cases

System

Episode Alerts is a mobile companion app for discovering and tracking television shows.

Assumptions

- TMDB is available when fresh show data is requested, or cached data is already stored on the device.
- The user has granted notification permissions if episode reminders are enabled.
- Firebase is configured only when optional cloud synchronization is being used.
- Local device storage is available for watchlist data, preferences, and cached content.

The Main Use Cases Are

- Discover TV Shows
- Search for TV Shows
- View Show Details
- Manage Watchlist
- Configure Preferences and Notifications
- Sync Data to Cloud

1. Use Case 1: Discover TV Shows

1.1 Primary Actor: User
1.2 Precondition: The app is open and the Home screen is available.
1.3 Scenario: The user browses the Home screen to explore featured and trending content.

1.3.1 Main Success Scenario

1. The user opens the Home screen.
2. The system loads featured, airing today, popular, and top-rated shows from TMDB or from cached data.
3. The system displays the show sections in a scrollable layout.
4. The user taps a show card.
5. The system opens the show details screen for the selected show.

1.3.2 Exception Scenario

- **2a.** If step 2 fails because the network is unavailable, the system shows stale cached data and a stale-data indicator.
- **4a.** If step 4 cannot be completed because the selected item is unavailable, the system stays on the Home screen.

2. Use Case 2: Search for TV Shows

2.1 Primary Actor: User
2.2 Precondition: The Search screen is available and the user can enter text.
2.3 Scenario: The user enters a query to find a specific TV show.

2.3.1 Main Success Scenario

1. The user opens the Search screen.
2. The user types a show title or keyword.
3. The system waits for a valid query and sends the request to TMDB.
4. The system displays the matching search results.
5. The user selects one of the results.
6. The system opens the show details screen for that result.

2.3.2 Exception Scenario

- **3a.** If step 3 fails because the request is too short, the system clears the results and waits for a valid search term.
- **3b.** If the network request fails, the system shows an error message and allows the user to retry.
- **4a.** If no results are returned in step 4, the system shows an empty state instead of a result list.

3. Use Case 3: View Show Details

3.1 Primary Actor: User
3.2 Precondition: The user has selected a show from Home, Search, or Watchlist.
3.3 Scenario: The user opens a show to review detailed information and explore seasons or episodes.

3.3.1 Main Success Scenario

1. The user opens a show details screen.

2. The system loads show metadata such as overview, rating, status, genres, and network information.
3. The system displays next episode information, last episode information, and available seasons.
4. The user opens a season to view its episode list.
5. The user returns to the show details screen or continues to another related action, such as adding the show to the watchlist.

3.3.2 Exception Scenario

- **2a.** If step 2 fails, the system falls back to cached details when available.
- **3a.** If a show does not have a next episode yet, the countdown area is hidden instead of showing invalid information.
- **4a.** If season data cannot be loaded, the system shows an error or empty state for that section only.

4. Use Case 4: Manage Watchlist

4.1 Primary Actor: User

4.2 Precondition: The user is viewing a show card, show details screen, or the Watchlist screen.

4.3 Scenario: The user adds, removes, or organizes shows in the watchlist.

4.3.1 Main Success Scenario

1. The user taps the watchlist action for a show.
2. The system saves the show locally in the watchlist store.
3. The system updates the watchlist screen and related counts immediately.
4. The system schedules release notifications when notifications are enabled.
5. The user later removes the show or clears the watchlist if needed.

4.3.2 Exception Scenario

- **1a.** If step 1 targets a show that is already in the watchlist, the system leaves the list unchanged and returns a duplicate-state response.
- **2a.** If step 2 fails because storage is unavailable, the system shows an error and keeps the previous watchlist state.
- **5a.** If the user clears the watchlist and later chooses undo, the system restores the last saved snapshot when available.

5. Use Case 5: Configure Preferences and Notifications

5.1 Primary Actor: User
5.2 Precondition: The Settings screen is available.
5.3 Scenario: The user changes app preferences such as theme, analytics, and notification behavior.

5.3.1 Main Success Scenario

1. The user opens the Settings screen.
2. The user changes the theme, analytics preference, or notification setting.
3. The system saves the preference locally and updates the app behavior.
4. If notifications are enabled, the system requests permission and schedules upcoming episode reminders.
5. The updated setting remains active the next time the app is opened.

5.3.2 Exception Scenario

- **2a.** If a preference update cannot be saved, the system shows an error and retains the previous value.
- **3a.** If analytics are disabled, the system stops recording usage events immediately.
- **4a.** If notification permission is denied in step 4, the system keeps notifications disabled and does not schedule reminders.

6. Use Case 6: Sync Data to Cloud

6.1 Primary Actor: Signed-in User
6.2 Precondition: Firebase is configured and the user is signed in.
6.3 Scenario: The system synchronizes watchlist data and user preferences with the cloud.

6.3.1 Main Success Scenario

1. The user signs in or triggers a synchronization event.
2. The system builds a payload from the watchlist, watch progress, watch history, and selected preferences.
3. The system uploads the payload to Firestore.
4. The system stores the last successful sync time.
5. The system repeats synchronization automatically when local data changes or when the app resumes.

6.3.2 Exception Scenario

- **1a.** If step 1 occurs while the device is offline, the system skips the upload and retries later.
- **3a.** If Firebase is not configured or the user is not signed in, the system keeps all data local.
- **5a.** If the cloud request fails, the system preserves local data and logs the sync failure for later retry.

Chapter 3

Code

3.1 Implementation Overview

The codebase is centered around a small set of services that support the app's main flow: fetching TV data, managing the user's watchlist, scheduling notifications, and persisting preferences. For this report, the two most important modules are TMDBService and WatchlistService because they drive the core data flow and the user's personalized tracking experience.

3.1.1 TMDBService

TMDBService is the app's main data layer for TV show content. It communicates with TMDB, normalizes show and episode data, and adds caching so the app can keep working even when the network is slow or unavailable. This service is central to the Home, Search, and Show Details screens because almost all show information comes through it.

```
1  import { TMDB_CONFIG, API_KEY } from '../../constants/Config';
2  import AsyncStorage from '@react-native-async-storage/async-
       storage';
3  import { reportError } from '@/app/utils/errorHandling';
4
5  export interface TVShow {
6    id: number;
7    name: string;
8    overview: string;
9    poster_path: string;
10   backdrop_path: string;
11   vote_average: number;
12   first_air_date: string;
13   next_episode_to_air?: Episode;
14   last_episode_to_air?: Episode;
```

```
  number_of_seasons: number;
  number_of_episodes?: number;
  status: string;
  genres: Genre[];
  networks: Network[];
  created_by: Creator[];
  seasons?: Season[];
  addedAt?: number;
}

export interface Episode {
  id: number;
  name: string;
  overview: string;
  still_path: string;
  air_date: string;
  episode_number: number;
  season_number: number;
  vote_average: number;
  runtime?: number;
}

export interface Season {
  id: number;
  name: string;
  overview: string;
  poster_path: string;
  air_date: string;
  season_number: number;
  episode_count: number;
  episodes?: Episode[];
}

export interface Genre {
  id: number;
  name: string;
}

export interface Network {
  id: number;
  name: string;
```

```
56    logo_path: string;
57  }
58
59  export interface Creator {
60    id: number;
61    name: string;
62    profile_path: string;
63  }
64
65  interface APIResponse<T> {
66    page?: number;
67    results: T[];
68    total_pages?: number;
69    total_results?: number;
70  }
71
72  interface CachedResponseEntry {
73    data: unknown;
74    expiresAt: number;
75    updatedAt: number;
76  }
77
78  const API_CACHE_STORAGE_KEY = '@EpisodeAlerts:tmdbApiCache';
79  const DEFAULT_CACHE_TTL_MS = 15 * 60 * 1000;
80  const MAX_CACHE_ENTRIES = 200;
81  const PERSIST_DEBOUNCE_MS = 400;
82  const MAX_STALE_RETENTION_MS = 7 * 24 * 60 * 60 * 1000;
83
84  class TMDBService {
85    private static instance: TMDBService;
86    private baseURL: string;
87    private headers: HeadersInit;
88    private memoryCache = new Map<string, CachedResponseEntry>();
89    private inFlightRequests = new Map<string, Promise<unknown>>();
90    private isCacheHydrated = false;
91    private hydratePromise: Promise<void> | null = null;
92    private persistTimer: ReturnType<typeof setTimeout> | null =
        null;
93    private staleFallbackUsed = false;
94    private lastCachedDataUpdatedAt: number | null = null;
95
```

```
96    private constructor() {
97      this.baseURL = TMDB_CONFIG.BASE_URL;
98      this.headers = {
99        'Authorization': `Bearer ${API_KEY}`,
100       'Content-Type': 'application/json',
101     };
102   }
103
104   public static getInstance(): TMDBService {
105     if (!TMDBService.instance) {
106       TMDBService.instance = new TMDBService();
107     }
108     return TMDBService.instance;
109   }
110
111   private normalizeEndpoint(endpoint: string): string {
112     const [path, queryString] = endpoint.split('?');
113     if (!queryString) {
114       return path;
115     }
116
117     const params = new URLSearchParams(queryString);
118     const sortedEntries = Array.from(params.entries()).sort((a, b)
          => {
119       if (a[0] === b[0]) {
120         return a[1].localeCompare(b[1]);
121       }
122       return a[0].localeCompare(b[0]);
123     });
124
125     const normalizedParams = new URLSearchParams();
126     sortedEntries.forEach(([key, value]) => {
127       normalizedParams.append(key, value);
128     });
129
130     return `${path}?${normalizedParams.toString()}`;
131   }
132
133   private getCacheTTL(endpoint: string): number {
134     const normalized = this.normalizeEndpoint(endpoint);
135
```

```
136     if (normalized.startsWith('/search/')) {
137       return 5 * 60 * 1000;
138     }
139
140     if (normalized.startsWith('/tv/airing_today')) {
141       return 10 * 60 * 1000;
142     }
143
144     if (normalized.startsWith('/tv/popular') || normalized.
          startsWith('/tv/top_rated')) {
145       return 30 * 60 * 1000;
146     }
147
148     if (/^\/tv\/\d+\/season\/\d+/.test(normalized)) {
149       return 6 * 60 * 60 * 1000;
150     }
151
152     if (/^\/tv\/\d+/.test(normalized)) {
153       return 60 * 60 * 1000;
154     }
155
156     return DEFAULT_CACHE_TTL_MS;
157   }
158
159   private buildCacheKey(endpoint: string): string {
160     return this.normalizeEndpoint(endpoint);
161   }
162
163   private async ensureCacheHydrated(): Promise<void> {
164     if (this.isCacheHydrated) {
165       return;
166     }
167
168     if (this.hydratePromise) {
169       await this.hydratePromise;
170       return;
171     }
172
173     this.hydratePromise = (async () => {
174       try {
175         const raw = await AsyncStorage.getItem(
```

```
            API_CACHE_STORAGE_KEY);
176         if (!raw) {
177           this.isCacheHydrated = true;
178           return;
179         }
180
181         const parsed = JSON.parse(raw) as Record<string,
               CachedResponseEntry>;
182         const now = Date.now();
183         Object.entries(parsed).forEach(([key, entry]) => {
184           if (!entry || typeof entry.expiresAt !== 'number') {
185             return;
186           }
187
188           const isFresh = entry.expiresAt > now;
189           const isWithinStaleRetention = now - entry.expiresAt <=
                MAX_STALE_RETENTION_MS;
190
191           if (isFresh || isWithinStaleRetention) {
192             this.memoryCache.set(key, entry);
193           }
194         });
195         this.isCacheHydrated = true;
196       } catch (error) {
197         reportError('TMDBService.ensureCacheHydrated', error, {
198           fallbackMessage: 'Failed to hydrate cached API data.',
199           trackAnalytics: false,
200         });
201         this.isCacheHydrated = true;
202       } finally {
203         this.hydratePromise = null;
204       }
205     })();
206
207     await this.hydratePromise;
208   }
209
210   private getCachedResponse<T>(cacheKey: string): { data: T | null
         ; isStale: boolean; updatedAt: number | null } {
211     const entry = this.memoryCache.get(cacheKey);
212     if (!entry) {
```

```
      return {
        data: null,
        isStale: false,
        updatedAt: null,
      };
    }

    if (entry.expiresAt <= Date.now()) {
      return {
        data: entry.data as T,
        isStale: true,
        updatedAt: entry.updatedAt,
      };
    }

    return {
      data: entry.data as T,
      isStale: false,
      updatedAt: entry.updatedAt,
    };
  }

  private trimCacheIfNeeded(): void {
    if (this.memoryCache.size <= MAX_CACHE_ENTRIES) {
      return;
    }

    const oldestEntries = Array.from(this.memoryCache.entries())
      .sort((a, b) => a[1].updatedAt - b[1].updatedAt)
      .slice(0, this.memoryCache.size - MAX_CACHE_ENTRIES);

    oldestEntries.forEach(([key]) => {
      this.memoryCache.delete(key);
    });
  }

  private setCachedResponse(cacheKey: string, data: unknown, ttlMs
     : number): void {
    const now = Date.now();
    this.memoryCache.set(cacheKey, {
      data,
```

```
253       expiresAt: now + ttlMs,
254       updatedAt: now,
255     });
256
257     this.trimCacheIfNeeded();
258     this.scheduleCachePersist();
259   }
260
261   private scheduleCachePersist(): void {
262     if (this.persistTimer) {
263       clearTimeout(this.persistTimer);
264     }
265
266     this.persistTimer = setTimeout(() => {
267       this.persistTimer = null;
268       this.persistCache().catch((error) => {
269         reportError('TMDBService.persistCache', error, {
270           fallbackMessage: 'Failed to persist cached API data.',
271           trackAnalytics: false,
272         });
273       });
274     }, PERSIST_DEBOUNCE_MS);
275   }
276
277   private async persistCache(): Promise<void> {
278     const serialized = Object.fromEntries(this.memoryCache.entries
          ());
279     await AsyncStorage.setItem(API_CACHE_STORAGE_KEY, JSON.
          stringify(serialized));
280   }
281
282   public async clearCache(): Promise<void> {
283     this.memoryCache.clear();
284     this.inFlightRequests.clear();
285     this.lastCachedDataUpdatedAt = null;
286     this.staleFallbackUsed = false;
287     await AsyncStorage.removeItem(API_CACHE_STORAGE_KEY);
288   }
289
290   public consumeStaleFallbackFlag(): boolean {
291     const staleFallbackUsed = this.staleFallbackUsed;
```

```
292     this.staleFallbackUsed = false;
293     return staleFallbackUsed;
294   }
295
296   public getLastCachedDataUpdatedAt(): number | null {
297     return this.lastCachedDataUpdatedAt;
298   }
299
300   private async fetchAPI<T>(endpoint: string): Promise<T> {
301     await this.ensureCacheHydrated();
302
303     const cacheKey = this.buildCacheKey(endpoint);
304     const cachedResponse = this.getCachedResponse<T>(cacheKey);
305     if (cachedResponse.data && !cachedResponse.isStale) {
306       this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
307       return cachedResponse.data;
308     }
309
310     const existingRequest = this.inFlightRequests.get(cacheKey);
311     if (existingRequest) {
312       return existingRequest as Promise<T>;
313     }
314
315     const requestPromise = (async () => {
316       try {
317         const url = `${this.baseURL}${endpoint}`;
318         const response = await fetch(url, {
319           method: 'GET',
320           headers: this.headers,
321         });
322
323         if (!response.ok) {
324           const errorData = await response.json().catch(() => null
                );
325           throw new Error(`HTTP error! status: ${response.status},
                 message: ${errorData?.status_message || 'Unknown
                error'}`);
326         }
327
328         const data = (await response.json()) as T;
329         this.setCachedResponse(cacheKey, data, this.getCacheTTL(
```

```
          endpoint));
330         this.lastCachedDataUpdatedAt = Date.now();
331         return data;
332       } catch (error) {
333         if (cachedResponse.data) {
334           this.staleFallbackUsed = true;
335           this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
336           return cachedResponse.data;
337         }
338
339         throw reportError('TMDBService.fetchAPI', error, {
340           fallbackMessage: 'Unable to load data from server.
                Please try again.',
341         });
342       } finally {
343         this.inFlightRequests.delete(cacheKey);
344       }
345     })();
346
347     this.inFlightRequests.set(cacheKey, requestPromise);
348
349     try {
350       return await requestPromise;
351     } catch (error) {
352       throw error;
353     }
354   }
355
356   public async getPopularTVShows(page = 1): Promise<APIResponse<
        TVShow>> {
357     return this.fetchAPI<APIResponse<TVShow>>(`/tv/popular?page=${
          page}`);
358   }
359
360   public async getTVShowDetails(id: number): Promise<TVShow> {
361     return this.fetchAPI<TVShow>(`/tv/${id}`);
362   }
363
364   public async searchTVShows(query: string, page = 1): Promise<
        APIResponse<TVShow>> {
365     return this.fetchAPI<APIResponse<TVShow>>(`/search/tv?query=${
```

```
        encodeURIComponent(query)}&page=${page}`);
  }

  public async getTopRatedTVShows(page = 1): Promise<APIResponse<
      TVShow>> {
    return this.fetchAPI<APIResponse<TVShow>>(`/tv/top_rated?page=
        ${page}`);
  }

  public async getTVShowsAiringToday(page = 1): Promise<
      APIResponse<TVShow>> {
    return this.fetchAPI<APIResponse<TVShow>>(`/tv/airing_today?
        page=${page}`);
  }

  public async getSeasonDetails(tvId: number, seasonNumber: number
      ): Promise<Season> {
    return this.fetchAPI<Season>(`/tv/${tvId}/season/${
        seasonNumber}`);
  }

  public getImageUrl(path: string, size: string): string {
    return `${TMDB_CONFIG.IMAGE_BASE_URL}/${size}${path}`;
  }
}

export default TMDBService.getInstance();
```

3.1.2 WatchlistService

WatchlistService manages the user's saved shows, watch progress, and watch history. It stores that data locally with AsyncStorage and can trigger cloud synchronization when Firebase is available. This service is important because it supports the app's personalized experience and connects the watchlist, notifications, and settings features.

```
import AsyncStorage from '@react-native-async-storage/async-
    storage';
import { TVShow, Episode } from './TMDBService';
import { reportError } from '@/app/utils/errorHandling';

const WATCHLIST_STORAGE_KEY = '@EpisodeAlerts:watchlist';
const WATCH_PROGRESS_STORAGE_KEY = '@EpisodeAlerts:watchProgress';
```

```
const WATCH_HISTORY_STORAGE_KEY = '@EpisodeAlerts:watchHistory';

export interface LastWatchedEpisode {
  showId: number;
  showName: string;
  seasonNumber: number;
  episodeNumber: number;
  episodeName: string;
  watchedAt: number;
}

export interface WatchHistoryEntry extends LastWatchedEpisode {
  id: string;
}

export interface WatchlistEntry {
  show: TVShow;
  addedAt: number;
}

export interface WatchlistSnapshot {
  watchlist: WatchlistEntry[];
  progressMap: Record<string, LastWatchedEpisode>;
  historyMap: Record<string, WatchHistoryEntry[]>;
}

class WatchlistService {
  private static instance: WatchlistService;

  private constructor() {}

  public static getInstance(): WatchlistService {
    if (!WatchlistService.instance) {
      WatchlistService.instance = new WatchlistService();
    }
    return WatchlistService.instance;
  }

  private triggerCloudSync(): void {
    void import('./CloudSyncService')
      .then((module) => module.default.syncToCloudIfSignedIn())
```

```
48        .catch((error) => {
49          reportError('WatchlistService.triggerCloudSync', error, {
50            fallbackMessage: 'Cloud sync could not be triggered.',
51          });
52        });
53    }
54
55    private normalizeWatchlist(rawWatchlist: unknown):
         WatchlistEntry[] {
56      if (!Array.isArray(rawWatchlist)) {
57        return [];
58      }
59
60      const now = Date.now();
61      const normalized: WatchlistEntry[] = [];
62
63      rawWatchlist.forEach((item, index) => {
64        if (item && typeof item === 'object' && 'show' in item) {
65          const entry = item as { show?: TVShow; addedAt?: number };
66          if (!entry.show || typeof entry.show.id !== 'number') {
67            return;
68          }
69
70          normalized.push({
71            show: {
72              ...entry.show,
73              addedAt: entry.addedAt,
74            },
75            addedAt: typeof entry.addedAt === 'number' ? entry.
                 addedAt : now,
76          });
77
78          return;
79        }
80
81        const show = item as TVShow;
82        if (!show || typeof show.id !== 'number') {
83          return;
84        }
85
86        const addedAt = show.addedAt ?? now - (rawWatchlist.length -
```

```
          index);
87        normalized.push({
88          show: {
89            ...show,
90            // Preserve original order for legacy arrays by creating
                  stable timestamps.
91            addedAt,
92          },
93          addedAt,
94        });
95      });
96
97      return normalized;
98    }
99
100   private async getWatchlistEntries(): Promise<WatchlistEntry[]> {
101     try {
102       const watchlistJson = await AsyncStorage.getItem(
             WATCHLIST_STORAGE_KEY);
103       const normalized = this.normalizeWatchlist(watchlistJson ?
             JSON.parse(watchlistJson) : []);
104       await this.saveWatchlistEntries(normalized);
105       return normalized;
106     } catch (error) {
107       reportError('WatchlistService.getWatchlistEntries', error, {
108         fallbackMessage: 'Unable to load watchlist entries.',
109       });
110       return [];
111     }
112   }
113
114   private async saveWatchlistEntries(entries: WatchlistEntry[]):
         Promise<void> {
115     await AsyncStorage.setItem(WATCHLIST_STORAGE_KEY, JSON.
          stringify(entries));
116   }
117
118   async getWatchlist(): Promise<TVShow[]> {
119     try {
120       const entries = await this.getWatchlistEntries();
121       return entries.map((entry) => ({
```

```
122         ...entry.show,
123         addedAt: entry.addedAt,
124       }));
125     } catch (error) {
126       reportError('WatchlistService.getWatchlist', error, {
127         fallbackMessage: 'Unable to load your watchlist.',
128       });
129       return [];
130     }
131   }
132
133   async getWatchlistSnapshot(): Promise<WatchlistSnapshot> {
134     const [watchlist, progressMap, historyMap] = await Promise.all
          ([
135       this.getWatchlistEntries(),
136       this.getProgressMap(),
137       this.getHistoryMap(),
138     ]);
139
140     return {
141       watchlist,
142       progressMap,
143       historyMap,
144     };
145   }
146
147   async restoreWatchlistSnapshot(
148     snapshot: WatchlistSnapshot,
149     options?: { skipCloudSync?: boolean },
150   ): Promise<boolean> {
151     try {
152       await this.saveWatchlistEntries(snapshot.watchlist);
153       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
             stringify(snapshot.progressMap));
154       await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
             stringify(snapshot.historyMap));
155
156       if (!options?.skipCloudSync) {
157         this.triggerCloudSync();
158       }
159
```

```
160       return true;
161     } catch (error) {
162       reportError('WatchlistService.restoreWatchlistSnapshot',
              error, {
163         fallbackMessage: 'Unable to restore watchlist snapshot.',
164       });
165       return false;
166     }
167   }
168
169   async addToWatchlist(show: TVShow): Promise<boolean> {
170     try {
171       const currentEntries = await this.getWatchlistEntries();
172
173       if (currentEntries.some((item) => item.show.id === show.id))
              {
174         return false;
175       }
176
177       const addedAt = Date.now();
178       const entry: WatchlistEntry = {
179         show: {
180           ...show,
181           addedAt,
182         },
183         addedAt,
184       };
185
186       const updatedWatchlist = [...currentEntries, entry];
187       await this.saveWatchlistEntries(updatedWatchlist);
188       this.triggerCloudSync();
189       return true;
190     } catch (error) {
191       reportError('WatchlistService.addToWatchlist', error, {
192         fallbackMessage: 'Unable to add this show to your
              watchlist.',
193       });
194       return false;
195     }
196   }
197
```

```
198   async removeFromWatchlist(showId: number): Promise<boolean> {
199     try {
200       const currentWatchlist = await this.getWatchlistEntries();
201       const updatedWatchlist = currentWatchlist.filter((show) =>
          show.show.id !== showId);
202
203       await this.saveWatchlistEntries(updatedWatchlist);
204       await this.clearLastWatchedEpisode(showId, { skipCloudSync:
          true });
205       await this.clearWatchHistory(showId, { skipCloudSync: true
          });
206       this.triggerCloudSync();
207       return true;
208     } catch (error) {
209       reportError('WatchlistService.removeFromWatchlist', error, {
210         fallbackMessage: 'Unable to remove this show from your
            watchlist.',
211       });
212       return false;
213     }
214   }
215
216   async isInWatchlist(showId: number): Promise<boolean> {
217     try {
218       const watchlist = await this.getWatchlistEntries();
219       return watchlist.some((show) => show.show.id === showId);
220     } catch (error) {
221       reportError('WatchlistService.isInWatchlist', error, {
222         fallbackMessage: 'Unable to check watchlist status.',
223         trackAnalytics: false,
224       });
225       return false;
226     }
227   }
228
229   async clearWatchlist(): Promise<boolean> {
230     try {
231       await AsyncStorage.removeItem(WATCHLIST_STORAGE_KEY);
232       await AsyncStorage.removeItem(WATCH_PROGRESS_STORAGE_KEY);
233       await AsyncStorage.removeItem(WATCH_HISTORY_STORAGE_KEY);
234       this.triggerCloudSync();
```

```
235       return true;
236     } catch (error) {
237       reportError('WatchlistService.clearWatchlist', error, {
238         fallbackMessage: 'Unable to clear your watchlist.',
239       });
240       return false;
241     }
242   }
243
244   private async getProgressMap(): Promise<Record<string,
        LastWatchedEpisode>> {
245     try {
246       const progressJson = await AsyncStorage.getItem(
            WATCH_PROGRESS_STORAGE_KEY);
247       return progressJson ? JSON.parse(progressJson) : {};
248     } catch (error) {
249       reportError('WatchlistService.getProgressMap', error, {
250         fallbackMessage: 'Unable to load watch progress.',
251       });
252       return {};
253     }
254   }
255
256   async getLastWatchedEpisodes(): Promise<Record<string,
        LastWatchedEpisode>> {
257     return this.getProgressMap();
258   }
259
260   private async getHistoryMap(): Promise<Record<string,
        WatchHistoryEntry[]>> {
261     try {
262       const historyJson = await AsyncStorage.getItem(
            WATCH_HISTORY_STORAGE_KEY);
263       return historyJson ? JSON.parse(historyJson) : {};
264     } catch (error) {
265       reportError('WatchlistService.getHistoryMap', error, {
266         fallbackMessage: 'Unable to load watch history.',
267       });
268       return {};
269     }
270   }
```

```
271
272   async getWatchHistory(showId: number): Promise<WatchHistoryEntry
          []> {
273     const historyMap = await this.getHistoryMap();
274     return (historyMap[String(showId)] || []).sort((a, b) => b.
          watchedAt - a.watchedAt);
275   }
276
277   private async appendWatchHistory(showId: number, showName:
          string, episode: Episode): Promise<void> {
278     const historyMap = await this.getHistoryMap();
279     const showKey = String(showId);
280     const existing = historyMap[showKey] || [];
281     const nextEntry: WatchHistoryEntry = {
282       id: `${showId}-${episode.season_number}-${episode.
            episode_number}-${Date.now()}`,
283       showId,
284       showName,
285       seasonNumber: episode.season_number,
286       episodeNumber: episode.episode_number,
287       episodeName: episode.name,
288       watchedAt: Date.now(),
289     };
290
291     const deduped = existing.filter(
292       (entry) =>
293         !(
294           entry.seasonNumber === episode.season_number &&
295           entry.episodeNumber === episode.episode_number &&
296           Math.abs(entry.watchedAt - nextEntry.watchedAt) < 60_000
297         ),
298     );
299
300     historyMap[showKey] = [nextEntry, ...deduped].slice(0, 50);
301     await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
          stringify(historyMap));
302   }
303
304   async getLastWatchedEpisode(showId: number): Promise<
          LastWatchedEpisode | null> {
305     const progressMap = await this.getProgressMap();
```

```
306     return progressMap[String(showId)] || null;
307   }
308
309   async setLastWatchedEpisode(showId: number, showName: string,
          episode: Episode): Promise<boolean> {
310     try {
311       const progressMap = await this.getProgressMap();
312       const watchedAt = Date.now();
313       progressMap[String(showId)] = {
314         showId,
315         showName,
316         seasonNumber: episode.season_number,
317         episodeNumber: episode.episode_number,
318         episodeName: episode.name,
319         watchedAt,
320       };
321
322       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
              stringify(progressMap));
323       await this.appendWatchHistory(showId, showName, episode);
324       this.triggerCloudSync();
325       return true;
326     } catch (error) {
327       reportError('WatchlistService.setLastWatchedEpisode', error,
              {
328         fallbackMessage: 'Unable to save watch progress.',
329       });
330       return false;
331     }
332   }
333
334   async clearLastWatchedEpisode(showId: number, options?: {
          skipCloudSync?: boolean }): Promise<boolean> {
335     try {
336       const progressMap = await this.getProgressMap();
337       delete progressMap[String(showId)];
338       await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
              stringify(progressMap));
339
340       if (!options?.skipCloudSync) {
341         this.triggerCloudSync();
```

```
342       }
343
344       return true;
345     } catch (error) {
346       reportError('WatchlistService.clearLastWatchedEpisode',
             error, {
347         fallbackMessage: 'Unable to clear watch progress.',
348       });
349       return false;
350     }
351   }
352
353   async clearWatchHistory(showId: number, options?: {
         skipCloudSync?: boolean }): Promise<boolean> {
354     try {
355       const historyMap = await this.getHistoryMap();
356       delete historyMap[String(showId)];
357       await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
             stringify(historyMap));
358
359       if (!options?.skipCloudSync) {
360         this.triggerCloudSync();
361       }
362
363       return true;
364     } catch (error) {
365       reportError('WatchlistService.clearWatchHistory', error, {
366         fallbackMessage: 'Unable to clear watch history.',
367       });
368       return false;
369     }
370   }
371 }
372
373 export default WatchlistService.getInstance();
```

Chapter 4

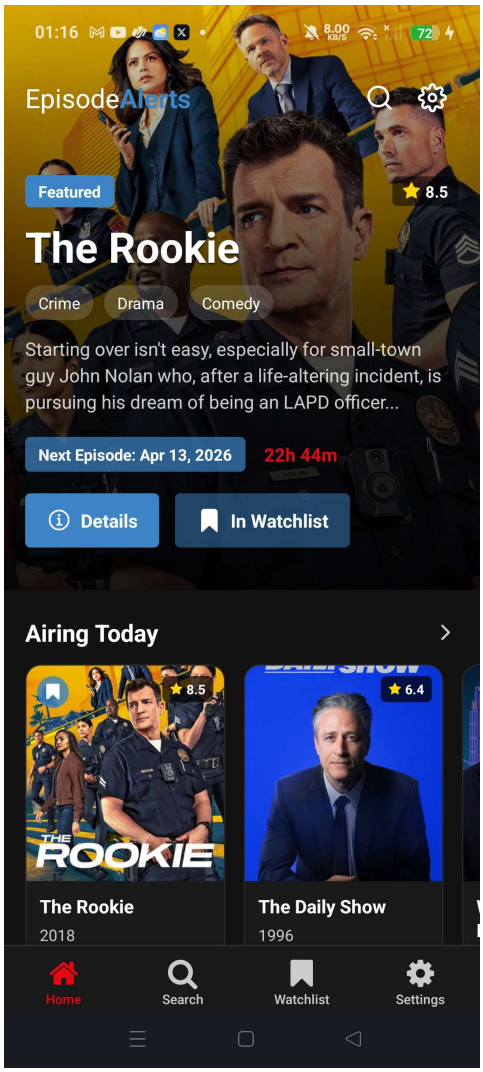
Results

4.1 Results Overview

The screenshots below show the implemented app in its final form, covering the main user flows: discovery, show details, watchlist management, analytics, and settings. Together, they demonstrate that the core screens are working as intended and that the app presents a consistent experience across its main features.

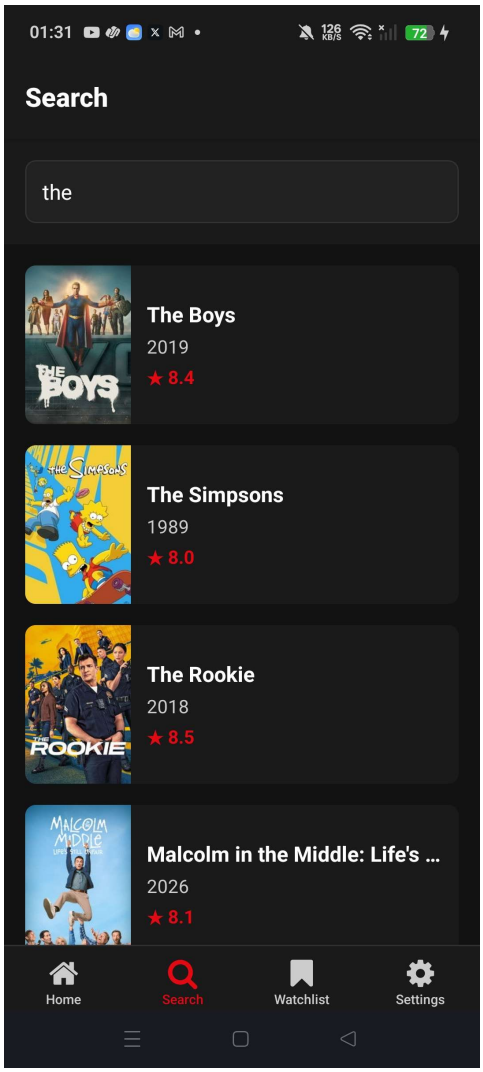
Source code is available on GitHub at EpisodeAlerts-MobileApp.

4.2 Application Screenshots

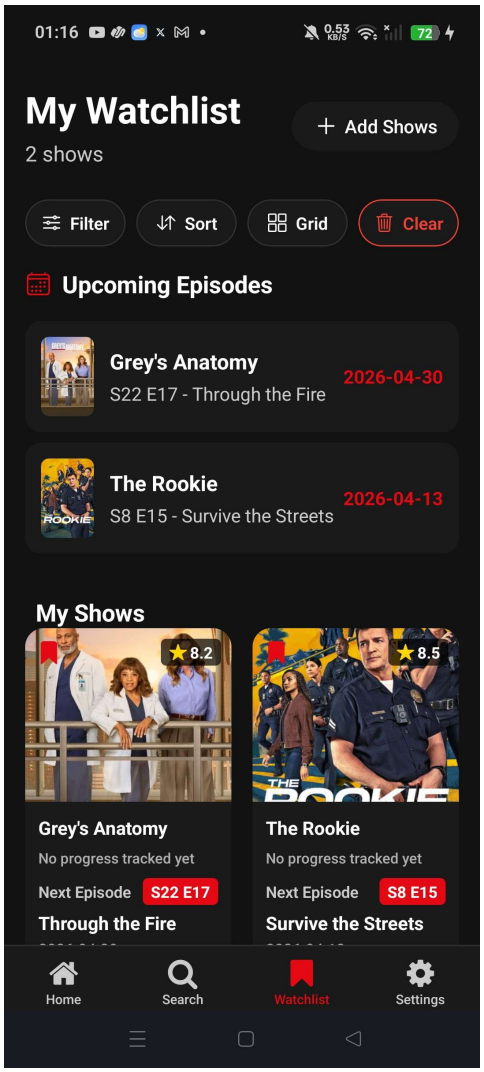


Home screen with featured content and category sections.

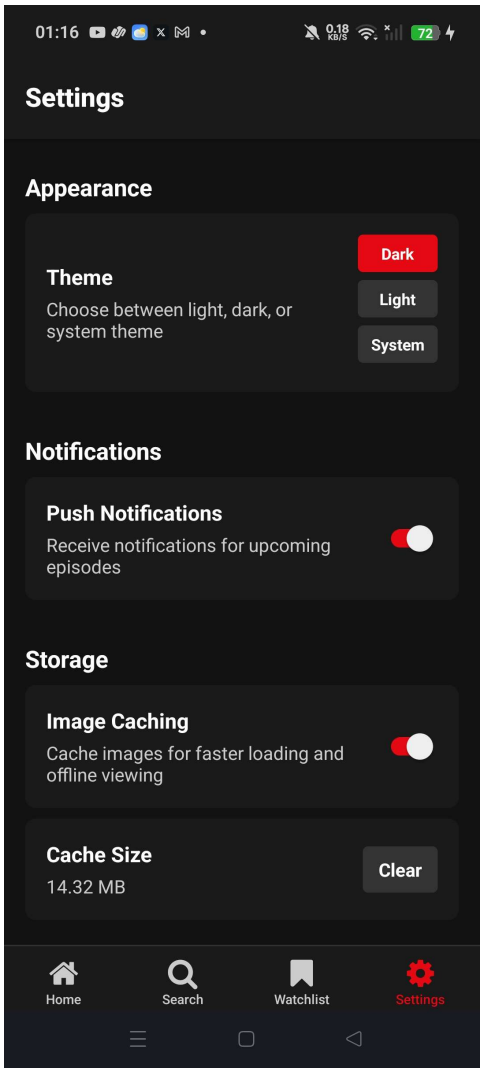
Show details screen with episode information and watchlist state.



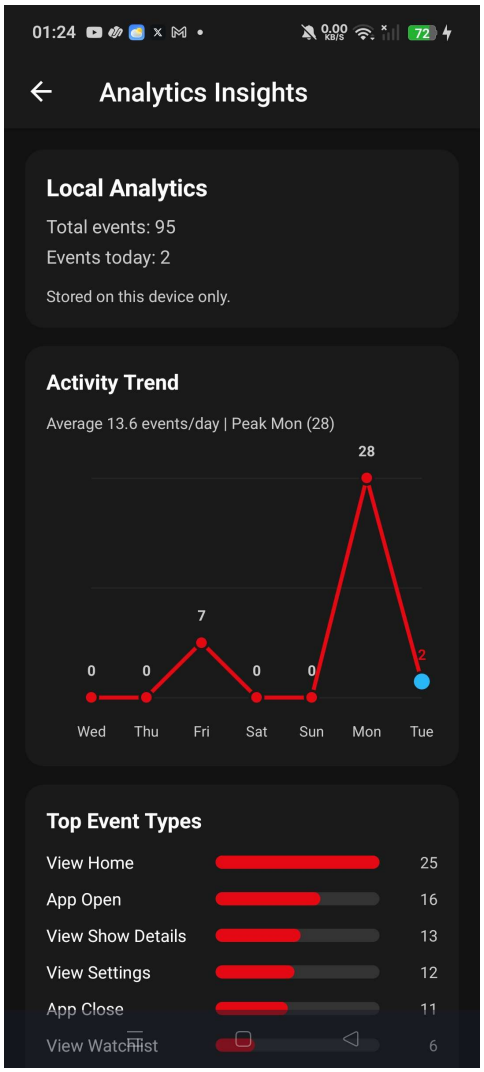
Search screen showing TV show search results.



Watchlist screen with upcoming episodes and saved shows.



Settings screen with theme, notifications, and caching options.



Analytics insights screen showing usage trends and event breakdowns.

4.3 Outcome Summary

The implemented build satisfies the core functional objectives defined in the SRS and demonstrates stable behavior across the primary user journeys.

- Discovery and search workflows successfully fetch and display TV data from TMDB, while preserving usability through cache and fallback behavior.
- Show details, season navigation, and next-episode context are presented consistently from Home, Search, and Watchlist entry points.
- Watchlist actions (add, remove, and update) persist correctly in local storage and reflect immediately in the user interface.
- Settings changes for theme, analytics, and notifications are retained and applied across app sessions.
- The analytics screen confirms event tracking visibility, and optional cloud-sync be-

havior aligns with the configured authentication state.

4.4 Conclusion

Episode Alerts delivers a complete and coherent mobile experience for TV discovery and tracking, with a clear architecture and reliable end-to-end flows. The final implementation is suitable for submission and provides a strong baseline for future enhancements such as richer personalization, expanded recommendation logic, and broader sync capabilities.